Software Requirements Specification

(SRS)

Web-Based Report Management Application

Backend: Python · Frontend: React · Database: Microsoft SQL Server

Team Size: 9 Members | Timeline: 1 Month

Version 1.0 · July 2026

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the requirements, architecture, and design plan for a Web-Based Report Management Application. It is intended to give the development team a shared, precise understanding of what the system must do and how the work will be divided so that a team of nine can build it within one month.

1.2 Objective

The objective is to build a secure, web-based application that allows authenticated users to manage and download reports, where each report is tied to a specific SQL query stored in the system.

The application acts as a controlled, self-service reporting layer between users and a Microsoft SQL Server database. An administrator writes a SQL query once and saves it under a friendly report name (for example, “Monthly Sales by Region”). Non-technical users can then log in, browse the available reports, run them, and download the results as PDF or CSV — without writing SQL or accessing the database directly.

This delivers three core benefits:

- **Saves time —** a query is written once and reused indefinitely instead of being re-requested each time.
- **Removes the technical barrier —** non-technical staff can pull their own data on demand.
- **Keeps data secure and controlled —** Role-Based Access Control (RBAC) ensures each role can perform only its permitted actions, and users never gain direct access to the raw database.

1.3 Scope

The system will provide the following capabilities:

- Secure user authentication with hashed passwords.
- User management with two roles (admin and viewer) governed by RBAC.
- Report management: defining reports, linking them to SQL queries, and editing or deleting them.
- Execution of predefined SQL queries against a Microsoft SQL Server database.
- Export of report results as downloadable PDF and CSV files.

Out of scope for this version: multi-tenant support, ad-hoc free-form querying by viewers, mobile native apps, and third-party single sign-on. These may be considered in future iterations.

1.4 Intended Audience

This document is intended for the nine project team members (frontend, database, and backend/security sub-teams), the project supervisor/instructor, and anyone reviewing or evaluating the system design.

1.5 Definitions & Abbreviations

Term	Meaning
SRS	Software Requirements Specification.
RBAC	Role-Based Access Control — permissions assigned by role rather than by individual.
Admin	A user role with full management rights over users and reports.
Viewer	A user role that can only run (execute) and download reports.
Report	A named definition that links a friendly name to a stored SQL query.
API	Application Programming Interface — how the frontend talks to the backend.
ORM	Object-Relational Mapper — maps Python objects to database tables.

2. Overall Description & System Architecture

The application follows a classic three-tier architecture. Each layer has a single, clear responsibility, which keeps the codebase organised and lets the three sub-teams work in parallel with minimal collisions.

2.1 Presentation Layer — Frontend (React)

What the user sees and interacts with: the login screen, report list, dashboard, and download controls. Built with React, it collects user input and displays results. It contains no business logic and never talks to the database directly — it communicates with the backend only through a REST API.

2.2 Application Layer — Backend (Python)

The brain of the system, built in Python. It receives requests from the React frontend, enforces security, and performs the work. It is responsible for:

- **Authentication & RBAC —** verifying who the user is and enforcing what their role may do.
- **Report Execution —** running a report's stored SQL query against SQL Server.
- **Export (PDF / CSV) —** converting query results into downloadable files.

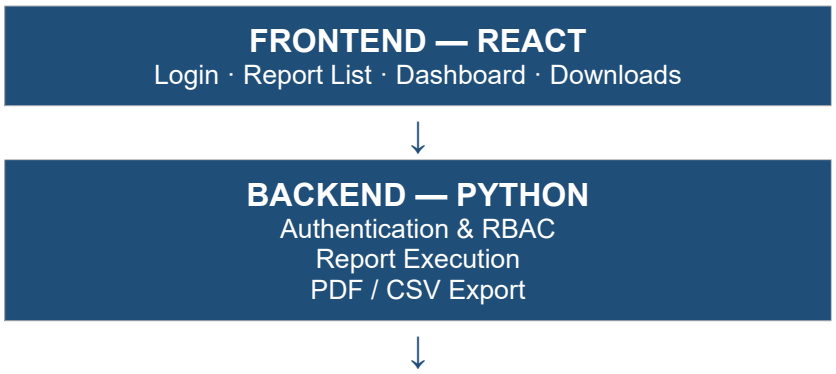
2.3 Data Layer — Microsoft SQL Server

Where all information is permanently stored. It holds two main tables — users (accounts, roles, hashed passwords) and reports (report names, descriptions, and their SQL queries). It stores and retrieves data but makes no decisions.

2.4 Request Flow

1. The React frontend sends a request (e.g., "run report #5") to the Python backend.
2. The backend authenticates the user and checks their role permissions.
3. If authorised, the backend executes the report's SQL query against SQL Server.
4. SQL Server returns the result set to the backend.
5. The backend formats the results (into PDF or CSV if requested).
6. The frontend displays the data or serves the file for download.

2.5 Architecture Diagram



DATABASE — MICROSOFT SQL SERVER

Users table

Reports table

3. Functional Requirements

These are the specific features the system must provide, taken directly from the project requirements.

3.1 Authentication

- The system shall provide a secure login form using a username and password.
- The system shall store passwords only as secure hashes, never in plain text.

3.2 User Management

- Admin users shall be able to create, edit, and delete user accounts.
- The system shall assign each user a role: admin or viewer.
- The system shall enforce RBAC: admins manage users and reports; viewers can only view and download reports.

3.3 Report Management

- Admin users shall be able to define reports by naming them and linking them to SQL queries.
- Admin users shall be able to edit or delete existing report definitions.
- Users shall be able to view a list of available reports and download results as PDF or CSV files.

3.4 Database Integration

- The system shall connect to a Microsoft SQL Server database.
- The system shall execute predefined SQL queries associated with reports.
- The system shall convert query results into downloadable PDF and CSV formats.

4. Non-Functional Requirements

Non-functional requirements describe how the system should behave and the quality standards it must meet, rather than the specific features it provides.

Category	Requirement
Security	Passwords stored as secure hashes (bcrypt); all access governed by RBAC; every permission enforced on the server side; app served over HTTPS; SQL queries vetted or parameterised to prevent injection.
Performance	Report queries and page loads should respond within a few seconds under normal load; the report list should stay responsive as the number of reports grows (pagination/indexing).
Usability	Clean, intuitive interface usable by non-technical staff; clear error and empty-state messages; actions a role cannot perform are hidden from that role.

Category	Requirement
Reliability & Availability	The system should handle failed queries gracefully without crashing; database backups taken regularly; no single failure should lose report definitions or user data.
Scalability	The architecture should support a growing number of users, reports, and stored queries without redesign; the three-tier separation allows each layer to scale independently.
Maintainability	Clean, layered, well-documented Python and React code following a consistent structure; version-controlled with Git so members can contribute safely.
Portability	The application should run consistently across all developer machines and deployment targets (optionally containerised with Docker).
Compatibility	The React frontend should work on modern browsers (Chrome, Firefox, Edge); the backend connects to standard Microsoft SQL Server.
Auditability	Key actions (login, create/edit/delete, execute, download) should be traceable through an audit log for accountability.

5. Tools & Technologies

The stack below covers every functional requirement using well-established, well-documented tools — important when nine people need to move quickly.

Layer / Concern	Technology	Purpose
Backend Language	Python	Core programming language for the server.
Backend Framework	Flask (or FastAPI)	Handles web server, routing, and REST API.
Security / Auth	Flask-JWT-Extended + bcrypt	Authentication, RBAC, and password hashing.
Data Access (ORM)	SQLAlchemy	Maps Python objects to database tables.
Database	Microsoft SQL Server	Stores users and report definitions.
DB Driver	pyodbc (ODBC Driver for SQL Server)	Connects Python to SQL Server.
PDF Export	ReportLab / FPDF2	Generates PDF files from query results.
CSV Export	Python csv module / pandas	Generates CSV files from query results.
Frontend	React	User interface (single-page app).
Charts	Chart.js	Dashboard charts (extra feature).
API Calls	Axios / Fetch	Frontend-to-backend communication.
Styling	Tailwind CSS / Bootstrap	Clean, responsive layout.
IDE	Visual Studio Code (VS Code)	Primary code editor for all members.
Version Control	Git + GitHub / GitLab	Team collaboration and code history.
API Testing	Postman	Testing backend endpoints.
Containerisation	Docker (optional)	Consistent environment for all members.

6. Database Schema & Access Control

6.1 users table

Column	Type	Description
id	INT, PK	Unique user ID.
username	VARCHAR	Login username.
password_hash	VARCHAR	Securely hashed password.
role	VARCHAR	'admin' or 'viewer'.

6.2 reports table

Column	Type	Description
id	INT, PK	Unique report ID.
name	VARCHAR	Report name.
description	TEXT	Optional description.
sql_query	TEXT	SQL query to execute.
created_by	INT, FK	Linked to users.id.

6.3 Access Control (RBAC) Mapping

The following table defines what each role is permitted to do. The backend enforces every one of these rules; the frontend additionally hides actions a role cannot perform.

Role	Action	Permission
Admin	Create Report / User	Allowed
Admin	Edit Report / User	Allowed
Admin	Delete Report / User	Allowed
Admin	Download Report	Allowed
Viewer	Create Report	Not Allowed
Viewer	Edit Report	Not Allowed
Viewer	Delete Report	Not Allowed
Viewer	Execute Report	Allowed
Viewer	Download Report	Allowed

7. Team Structure & Responsibilities

The nine-member team is divided into three sub-teams aligned with the three architecture layers. Each sub-team owns a layer end-to-end, which minimises overlap and makes progress easy to track.

Sub-Team	Members	Owns
Frontend (React)	3	Presentation layer — all user-facing screens & dashboard.
Database	2	Data layer — schema, SQL Server setup, and data access.
Backend / Security (Python)	4	Application layer — auth, RBAC, report execution, export.

7.1 Frontend Team — React (3 members)

Responsible for everything the user sees and interacts with, built to consume the backend REST API.

- **Member F1 — Authentication & Layout:** login page, client-side session/token handling, route guarding by role, and the overall app shell/navigation.
- **Member F2 — Admin Screens:** user management UI (create/edit/delete users) and report management UI (define/edit/delete report definitions).
- **Member F3 — Viewer Screens & Dashboard:** report list, report execution view, download controls, and the charts/dashboard (extra feature).

7.2 Database Team (2 members)

Responsible for the data layer and ensuring the backend can reliably read and write data.

- **Member D1 — Schema & Setup:** design and create the users and reports tables, keys, constraints, SQL Server installation, and seed/sample data.
- **Member D2 — Data Access & Optimisation:** SQLAlchemy models, safe execution of report queries, indexing, backups, and the audit-log table (extra feature).

7.3 Backend / Security Team — Python (4 members)

Responsible for the application layer: all business logic, security, and the report pipeline.

- **Member B1 — Authentication:** login endpoint, password hashing (bcrypt), JWT/session handling, and security configuration.
- **Member B2 — RBAC & User Management API:** role definitions, permission enforcement on every endpoint, and the admin-only user-management endpoints.
- **Member B3 — Report Management & Execution:** endpoints to define/edit/delete reports and to execute a report's SQL query safely against SQL Server.
- **Member B4 — Export & Integrations:** PDF and CSV generation, plus extra features (scheduled email delivery and export formatting).

Critical dependency: RBAC is the backbone that Authentication, the Backend, and the Frontend all depend on. Its interface (roles, permissions, and API contract) should be agreed and stubbed in week 1 so no sub-team is left waiting.

8. Additional Features (Enhancements)

These enhancements reuse the core pipeline and turn the project from a basic CRUD app into something that feels like a real product. They are ordered by value-for-effort and should only begin once the core is demo-able.

8.1 Dashboard with Charts

Feeds report results into visual charts (bar, line, pie) instead of plain tables, so trends and comparisons are understood at a glance. It reuses existing query results, making it the biggest visual upgrade for the effort. Charts should be optional per report and matched to the data.

Owner: Frontend F3 (with data shaping from Backend B3). Tool: Chart.js.

8.2 Scheduled Reports + Email Delivery

Lets a user schedule a report to run automatically (daily/weekly) and receive the PDF/CSV by email. It maps naturally onto the existing execute-and-export pipeline.

Owner: Backend B4. Tools: APScheduler / Celery for scheduling, smtplib / SendGrid for email.

8.3 Audit Log

Records who ran, created, edited, or downloaded what, and when. A small table plus logging that makes the RBAC story feel complete and enterprise-grade by adding accountability.

Owner: Database D2 (table) + Backend B2 (logging hooks).

8.4 Parameterised Reports

Lets a report accept safe inputs (e.g., a date range or region) that fill placeholders in the SQL. This is both a useful feature and a direct defence against SQL injection, since inputs are parameterised rather than concatenated into the query.

Owner: Backend B3 (with Frontend F3 for input controls).

8.5 Nice-to-Haves (if time allows)

- Search, sort, and pagination on the report list.
- “Favourites” or “recently run” section per user.
- Excel (.xlsx) export alongside PDF/CSV (openpyxl).
- Dark mode / clean UI theme for extra polish.
- Dockerise the app for one-command setup and demos.

9. Security Considerations

- **Password hashing —** store only bcrypt hashes; never plain-text passwords.
- **SQL injection —** RBAC controls who can run a report, but not whether the stored SQL is safe. Keep report queries vetted or parameterised so a malicious or malformed query cannot cause damage even when run by an authorised admin.
- **Least privilege —** the database account the app uses should have only the permissions it needs (ideally read-only for report execution).

- **Transport security —** serve the app over HTTPS so credentials and data are encrypted in transit.
- **Server-side enforcement —** never rely on the frontend hiding buttons; every permission is checked again on the backend.

10. One-Month Timeline

Guiding principle: get the core working and demo-able first, then add enhancements. A polished core beats a pile of half-finished features.

Week	Focus	Key Deliverables
Week 1	Setup & Foundations	Repos & VS Code environments set up; SQL Server schema created; RBAC contract agreed & stubbed; login/authentication working; project skeletons for React and Python.
Week 2	Core Features	User management (CRUD); report definition (CRUD); report execution against SQL Server; RBAC enforced on all endpoints. Core demo-able by end of week.
Week 3	Export & Key Enhancements	PDF and CSV export complete; dashboard with charts; parameterised reports; audit log. Integrate React frontend with Python backend end-to-end.
Week 4	Enhancements, Testing & Polish	Scheduled email reports; nice-to-haves as time allows; full testing (especially RBAC); bug fixes; UI polish; documentation; final demo prep.

Milestone check: By the end of Week 2 the core application should be fully functional and demonstrable. Everything in Weeks 3–4 builds on that stable base.